

NAME: Vibhum Bhattad

PRN:202501050037


```
print(f"{momentum :.2f}kgm/s")
```

Test case 1

EDS pratical 1.1.2

Write a Python program that accepts an integer n as input. Depending on the number of digits in n .

Constraints:

$1 \leq n \leq 999$

Input Format:

The input consists of a single integer n .

Output Format:

If n is a single-digit number, print its square.

If n is a two-digit number, print its square root (rounded to two decimal places).

If n is a three-digit number, print its cube root (rounded to two decimal places).

Else print "Invalid".

Average time

0.004 s

3.75 ms

Maximum time

0.006 s

6.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 6 ms Debug

Expected output

Actual output

5.0

5.0

10.0

10.0

50.00kgm/s

50.00kgm/s

Test case 2 4 ms Debug

Expected output

Actual output

2.3

2.3

4.8

4.8

11.04kgm/s

11.04kgm/s

9	
81↵	
Test case 2	
166	
5.50↵	
Test case 3	
24	
4.90↵	
Test case 4	
3456	
Invalid↵	
import math	
number = int(input())	
if 1<= number <=9: print(number **2)	
elif 10<= number <=99: print(f"{math.sqrt(number):.2f}")	
elif 100<= number <=999: print(f"{math.cbrt(number):.2f}")	
else: print("Invalid")	

Write a Python program to calculate number of days between two dates.

Input Format:

The first line contains the first date in the format .

The second line contains the second date in the format .

Output Format:

An integer representing the number of days between the given dates.

Note:

Average time
0.002 s
2.29 ms

Maximum time
0.005 s
5.00 ms

✓ 4 out of 4 shown test case(s) passed

✓ 3 out of 3 hidden test case(s) passed

✓ Test case 1 5 ms Debug

Expected output

9

81

Actual output

9

81

✓ Test case 2 4 ms Debug

Expected output

166

5.50

Actual output

166

5.50

✓ Test case 3 2 ms Debug

Expected output

24

4.90

Actual output

24

4.90

✓ Test case 4 1 ms Debug

Expected output

3456

Invalid

Actual output

3456

Invalid

difference=(d2 - d1)

```
print(difference.days)
```

```
date1= input().strip()
```

```
date2= input().strip()
```

```
d1= datetime.strptime(date1,"%Y-%m-%d")
```

```
d2= datetime.strptime(date2,"%Y-%m-%d")
```

The first date should always be considered earlier than the second date.	
Sample Test Cases	
Test case 1	
2014-07-02	
2014-11-02	
123🚩	
Test case 2	
2014-07-02	
2014-07-11	
9🚩	

```
from datetime import datetime
```



```
print(reversed_num)
```

```
reversed_num = int(str(num)[::-1])
```

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:
The input is a single integer.

Output Format:
The output prints a single integer, which is the reversed number.

Sample Test Cases

Test case 1

5367	
7635 ↵	

Test case 2

0132	
231 ↵	

```
num = int(input())
```

Average time
0.005 s
5.50 ms

Maximum time
0.013 s
13.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 13 ms Debug

Expected output

2014-07-02

2014-11-02

123

Actual output

2014-07-02

2014-11-02

123

Test case 2 3 ms Debug

Expected output

2014-07-02

2014-07-11

9

Actual output

2014-07-02

2014-07-11

9

1.1.5. Multiplication Table

03:25

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

Print the multiplication table for the given number in the format:

<number> x <multiplier> = <result>

Note:

The character "x" is used in the output to represent multiplication.
Refer to the visible test-cases for more clarity.

Sample Test Cases

Test case 1

8	
8 x 1 = 8	

Average time

0.002 s

1.80 ms

Maximum time

0.002 s

2.00 ms

2 out of 2 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 2 ms Debug

Expected output

5367

7635

Actual output

5367

7635

Test case 2 2 ms Debug

Expected output

0132

231

Actual output

0132

231

for i in range(1,11):

```
print(f"{num} x {i} = {num*i}")
```

8 · x · 2 = · 16↵	
8 · x · 3 = · 24↵	
8 · x · 4 = · 32↵	
8 · x · 5 = · 40↵	
8 · x · 6 = · 48↵	
8 · x · 7 = · 56↵	
8 · x · 8 = · 64↵	
8 · x · 9 = · 72↵	
8 · x · 10 = · 80↵	

```
num = int(input())
```


1.2.1. Pass or Fail

27:50

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

If the aggregate percentage is greater than 75, the grade is Distinction.

If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.

If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.

Average time

0.003 s

3.00 ms

Maximum time

0.006 s

6.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

6 ms

Debug

Expected output

Actual output

8

8

8 · x · 1 = 8

8 · x · 1 = 8

8 · x · 2 = 16

8 · x · 2 = 16

8 · x · 3 = 24

8 · x · 3 = 24

8 · x · 4 = 32

8 · x · 4 = 32

8 · x · 5 = 40

8 · x · 5 = 40

8 · x · 6 = 48

8 · x · 6 = 48

8 · x · 7 = 56

8 · x · 7 = 56

8 · x · 8 = 64

8 · x · 8 = 64

8 · x · 9 = 72

8 · x · 9 = 72

8 · x · 10 = 80

8 · x · 10 = 80

Test case 2

3 ms

Debug

Expected output

Actual output

5

5

5 · x · 1 = 5

5 · x · 1 = 5

5 · x · 2 = 10

5 · x · 2 = 10

5 · x · 3 = 15

5 · x · 3 = 15

5 · x · 4 = 20

5 · x · 4 = 20

5 · x · 5 = 25

5 · x · 5 = 25

5 · x · 6 = 30

5 · x · 6 = 30

5 · x · 7 = 35

5 · x · 7 = 35

5 · x · 8 = 40

5 · x · 8 = 40

5 · x · 9 = 45

5 · x · 9 = 45

5 · x · 10 = 50

5 · x · 10 = 50

If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

The first input will be an integer , the number of courses.

The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

Print the aggregate percentage (formatted to two decimal places).

Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

"Fail".

Note:

Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

Test case 1

4	
55 65 78 89	
Aggregate Percentage: 71.75	
Grade: First Division	

Test case 2

5	
56 78 97 86 93	
Aggregate Percentage: 82.00	
Grade: Distinction	

Test case 3

l

s

e

:

p

r

i

n

t

(

"

F

a

i

l

"

)

if(percentage>75):

```
grade = "Distinction"
```

```
elif(percentage>=60):
```

```
elif(percentage>=50):
```

```
grade = "First Division"
```

```
grade = "Second Division"
```

```
percentage = sum(marks)/n
```

```
marks = list(map(int,input().split()))
```

```
if any(mark < 40 for mark in marks):
```


5	
99 85 43 97 34	
Fail	

Test case 4

3	
55 54 53	
Aggregate Percentage: 54.00%	
Grade: Second Division	

Test case 5

5	
40 45 42 48 41	
Aggregate Percentage: 43.20%	
Grade: Third Division	

n = int(input())

l

s

e

:

```
print(f"Grade: {grade}")
```

```
grade = "Third Division"
```

```
print(f"Aggregate Percentage: {percentage:.2f}")
```

1.2.2. Fibonacci Series

14:07

Write a Python program that uses recursion to print the first `terms` of the Fibonacci series.

Input Format:

A single integer `terms` representing the number of terms to generate.

Output Format:

A single line containing the first `terms` of the Fibonacci sequence, separated by spaces.

Sample Test Cases

Test case 1

Average time
0.003 s
3.20 ms

Maximum time
0.005 s
5.00 ms

5 out of 5 shown test case(s) passed
5 out of 5 hidden test case(s) passed

Test case 1
Expected output
4
35 45 78 89
Aggregate Percentage: 75.75
Grade: First Division

Actual output
4
35 45 78 89
Aggregate Percentage: 75.75
Grade: First Division

Test case 2
Expected output
5
34 78 97 86 93
Aggregate Percentage: 82.00
Grade: Distinction

Actual output
5
34 78 97 86 93
Aggregate Percentage: 82.00
Grade: Distinction

Test case 3
Expected output
5
99 85 43 37 34
Fail

Actual output
5
99 85 43 37 34
Fail

Test case 4
Expected output
3
35 54 53
Aggregate Percentage: 54.00
Grade: Second Division

Actual output
3
35 54 53
Aggregate Percentage: 54.00
Grade: Second Division

else:

if n == 1:

return 1

return 0

elif n == 2:

n = int(input())

write the code..

for i in range(1, n + 1):

print(fibonacci(i), end=" ")

return fibonacci(n-1) + fibonacci(n-2)

5	
0•1•1•2•3•	
Test case 2	
15	
0•1•1•2•3•5•8•13•21•34•55•89•144•233•377•	
def fibonacci(n):	

1.2.3. Pattern - 1

05:40

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

The input is an integer, representing the number of rows in the pattern.

Output Format

The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases

Test case 1

5	
*.	
..	
..*.	
..*.*.	

Average time

0.007 s

7.00 ms

Maximum time

0.017 s

17.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

4 ms

Debug

Expected output

Actual output

5

5

0 · 1 · 1 · 2 · 3 ·

0 · 1 · 1 · 2 · 3 ·

Test case 2

3 ms

Debug

Expected output

Actual output

15

15

0 · 1 · 1 · 2 · 3 · 5 · 8 · 13 · 21 · 34 · 55 · 89 · 144 · 233 · 377 ·

0 · 1 · 1 · 2 · 3 · 5 · 8 · 13 · 21 · 34 · 55 · 89 · 144 · 233 · 377 ·

```
print('* ' * i)
```

```
n =int(input())
```

```
for i in range(1,n + 1):
```

..*.*.*.*.*	
Test case 2	
4	
.	
..*	
..*.*	
..*.*.*	

1.2.4. Pattern - 2

13:43

Write a Python program to print a right-angled triangle pattern of numbers.

Average time

0.003 s

2.50 ms

Maximum time

0.004 s

4.00 ms

2 out of 2 shown test case(s) passed

4 out of 4 hidden test case(s) passed

Test case 1

4 ms

Debug

Expected output

Actual output

5

5

* , *

* , *

* , * , *

* , * , *

* , * , * , *

* , * , * , *

* , * , * , * , *

* , * , * , * , *

Test case 2

2 ms

Debug

Expected output

Actual output

4

4

* , *

* , *

* , * , *

* , * , *

* , * , * , *

* , * , * , *

* , * , * , * , *

* , * , * , * , *

Input Format:

The input is an integer, representing the number of rows in the pattern.

Output Format:

The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

Refer to the displayed test cases for the sample pattern.

Test case 1

5	
1 •	
1 • 2 •	
1 • 2 • 3 •	
1 • 2 • 3 • 4 •	
1 • 2 • 3 • 4 • 5 •	

Test case 2

8	
1 •	
1 • 2 •	
1 • 2 • 3 •	
1 • 2 • 3 • 4 •	
1 • 2 • 3 • 4 • 5 •	
1 • 2 • 3 • 4 • 5 • 6 •	
1 • 2 • 3 • 4 • 5 • 6 • 7 •	
1 • 2 • 3 • 4 • 5 • 6 • 7 • 8 •	

n=int(input())

r

i

n

t

(

)

p

r

i

n

t

(

j

,

e

n

d

=

"

"

)

for j in range(1,i+1):

for i in range(1,n + 1):

Average time
0.004 s
3.75 ms

Maximum time
0.006 s
6.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 6 ms Debug

Expected output

Actual output

5

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

Test case 2 3 ms Debug

Expected output

Actual output

8

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

1 2 3 4 5 6

1 2 3 4 5 6 7

1 2 3 4 5 6 7 8

velocity = float(input())

momentum = mass*velocity

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula:

where:

is the mass of the object (in kilograms).

is the velocity of the object (in meters per second).

Input Format:

A single floating-point number representing the mass of the object in kilograms.

A single floating-point number representing the velocity of the object in meters per second.

Output Format:

The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Test case 1	
5.0	
10.0	
50.00kgm/s	
Test case 2	
2.3	
4.8	
11.04kgm/s	

```
mass = float(input())
```